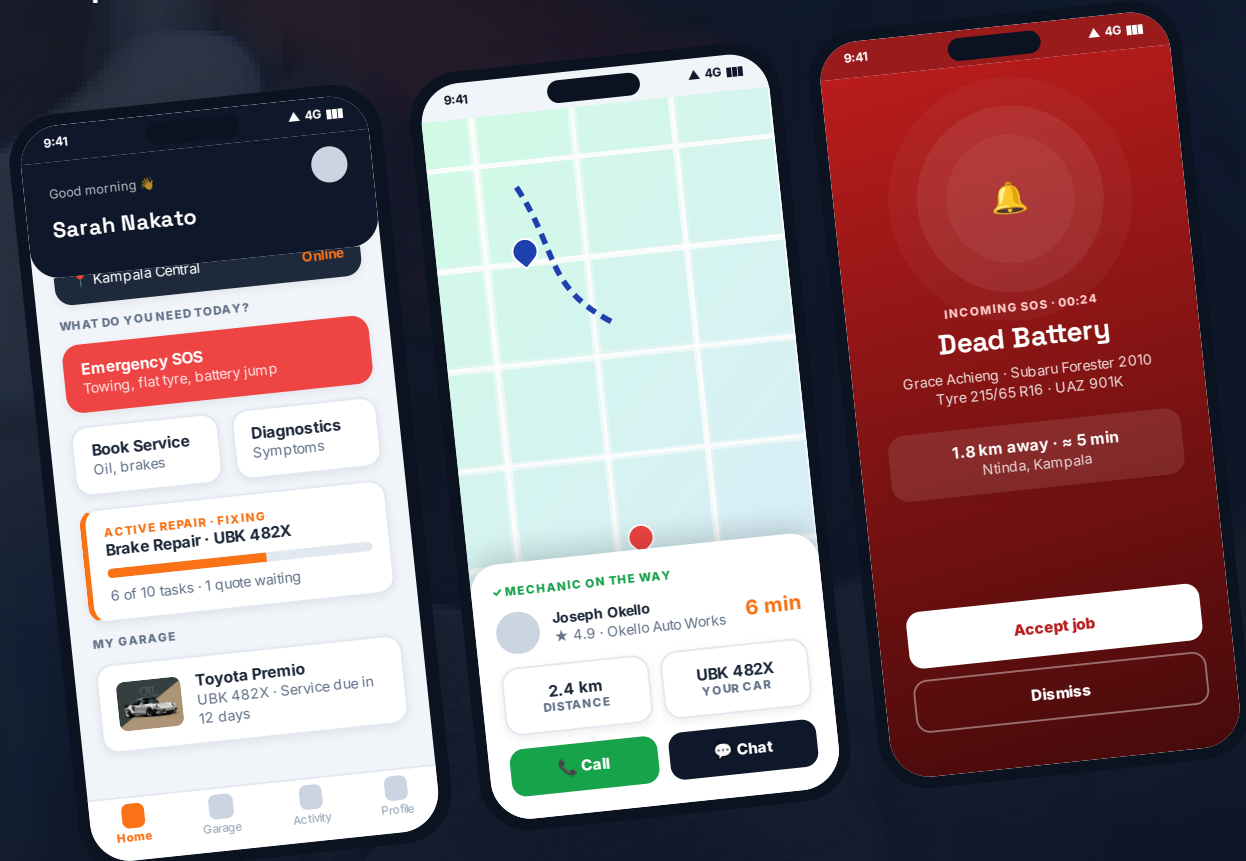




MyCarRepair



PRODUCT & TECHNICAL BLUEPRINT · MOBILE

Mobile App Blueprint

How to turn the MyCarRepair web platform into native Android & iOS apps for car owners and mechanics — reusing the **same data models**, business rules and backend.

Project

MyCarRepair (Kampala, Uganda)

Product owner

Kwagala Ian Magumba

Version · Date

1.0 · 24 Sep 2026

Platforms

Android first · iOS

Stack

React Native · Expo · TypeScript

Companion document

Web Platform Documentation

Contents

1	Vision, goals & scope
2	Users, journeys & product decisions
3	Feature scope — web parity & native enhancements
4	Target architecture & technology stack
5	Data model — same models as the web
6	Backend work: API v1, authentication, dispatch
7	API contract (v1)
8	Real-time events & push notifications
9	Information architecture & navigation
10	Screen specifications with mock-ups
11	Native capabilities, permissions & offline
12	Design system for mobile
13	Security, privacy & non-functional requirements
14	Project structure, testing, CI/CD & release
15	Roadmap, team & definition of done
A	Appendix — TypeScript models, payloads, SQL

Principle of this blueprint

The app is a **new client**, not a new system. The PostgreSQL schema, the job lifecycle, the fee rules and the real-time event names used by the web platform stay the same, so the web app, the admin portal and the mobile apps can run side by side against one database. Everything the web does today is listed in §3 with its mobile equivalent. Mock-ups in §10 are placed next to the matching web screens.

1 Vision, goals & scope

MyCarRepair was designed from the start as a **mobile-first** service for Kampala's smartphone-first users (dissertation §1.6). It shipped as a web app for the academic MVP. This blueprint describes the native mobile app it was meant to be. Native gives the product what a browser can't do reliably: **push notifications** when the app is closed, **background location** for mechanics on the way, a **native camera** for evidence photos, **full-screen SOS alerts**, **offline queues**, and a presence on the Play Store where Ugandan users look for apps.

Goals

- Feature parity with every owner and mechanic function of the web platform (§3).
- SOS reaches a nearby mechanic in **under 2 s**, even if the mechanic's app is in the background.
- Owners approve parts from a notification in **2 taps**.
- Works on entry-level Android phones (2 GB RAM, Android 8+) and on 3G.
- Reuse the existing database and backend — **no changes to the existing data models**.

Non-goals for the MVP

- Admin console in the app (stays on the web: desktop, low-frequency use).
- In-app payments (MTN MoMo / Airtel Money): Phase 2.
- In-app chat, SMS SOS fallback: Phase 2.
- AI diagnostics / OBD-II: Phase 3.
- Rewriting the backend in another language or framework.

1.1 What stays and what changes

Area	Stays the same	Changes / is added
Data models	users, vehicles, jobs, job_checklists, parts_quotes, reviews, system_config — same columns, types, enums	Nothing changes in these tables. Push tokens are the only new storage need (§5.3).
Business rules	UGX 50,000 service fee, first-come claim, 1 review per job, parts approved before billing, pending mechanic approval	Implement FR-03 geo-dispatch (nearest 5 ≤10 km) using the existing lat/long columns; SOS cancel → cancelled
Backend	Node.js + Express + Socket.io + PostgreSQL in the same repo	New JSON API <code>/api/v1</code> with JWT, role and ownership guards, cloud media storage, push service
Real-time	Event names (<code>new_job_pushed</code> , <code>job_taken</code> , <code>task_update</code> ...)	Server-assigned rooms; events scoped to <code>job_<id></code> / <code>user_<id></code> ; push when the app is backgrounded
Front-end	Brand colours, Grease-Proof UI, flows and copy	React Native screens with native navigation, camera, maps, haptics
Admin	Web admin portal	Gains "mechanic approved" push trigger; no mobile admin

2 Users, journeys & product decisions

Sarah — car owner

34, accountant in Kampala, drives a 2008 Toyota Premio. Has been overcharged before and does not trust roadside garages. Uses WhatsApp, MoMo and Bolt daily on a mid-range Android.

Needs: fast help in a breakdown, to see and approve every part, to know what is happening to her car.

Joseph — mechanic

29, runs Okello Auto Works in Wandegeya and does mobile call-outs. Gets clients through referrals. Hands are often greasy; works outdoors in bright sun.

Needs: a steady stream of nearby jobs, quick quoting with photos, proof of work, visible reputation, fast payment.

Admin — operations

Verifies mechanics, monitors live jobs, handles disputes and suspensions, switches on maintenance mode.

Needs: a desktop dashboard (already built), mechanic documents to review, push to tell mechanics they are approved.

2.1 Key journeys (happy paths)

Journey	Steps in the app	Target
Emergency breakdown	Open app → SOS → pick issue (GPS auto) → see mechanic + ETA on map → call → track repair → approve parts → receipt → rate	≤ 3 taps to send SOS
Scheduled service	Home → Book → service + date → mechanic accepts (push) → tracker → approve parts → receipt → rate	≤ 60 s to book
Mechanic SOS job	Online → full-screen alert → Accept → navigate (live location shared) → "Reached car" → checklist → quote → finish → earnings updated	Accept in ≤ 2 taps
Mechanic onboarding	Sign up → garage details → upload ID/permit → under review → push "verified" → go online	Approval < 24 h

2.2 Product decisions

Decision	Choice	Rationale
One app or two?	One app, two role modes (owner / mechanic)	Matches the web (single login with role toggle), one codebase and one store listing, simpler releases. Could split later if store positioning requires it.
Framework	React Native + Expo (TypeScript)	Same language as the backend and the dissertation's React choice; mature maps/camera/push modules; EAS builds without owning a Mac for Android; OTA fixes. Flutter was considered: equally capable, but adds Dart and duplicates model code.
Platform order	Android first , iOS in the same codebase	Android has a very large majority of the Ugandan market; iOS follows at little extra cost.

Decision	Choice	Rationale
Auth	JWT access (24 h) + refresh (30 d)	Follows dissertation FR-02/NFR-04; cookies are awkward in native apps. Web keeps cookie sessions.
Media	Object storage with signed URLs	Local disk does not scale and exposes files publicly; mobile uploads need resumable, CDN-served images.
Admin	Stays web	Desktop workflow; the existing portal covers it.
Language	English first; i18n-ready for Luganda / Swahili	Kampala users are comfortable in English; localisation widens reach.

3 Feature scope — web parity & native enhancements

Every row is an existing web feature unless its phase says otherwise. Screen IDs refer to §10.

Feature (web today)	Screen	Native enhancement	Phase
Sign up / login (owner, mechanic)	A1–A4	Biometric unlock, phone-number login, remember device	MVP
Owner dashboard	O1	Active-job card, push-driven toasts, pull-to-refresh	MVP
Virtual garage (add/view/edit/delete, ≤5 photos)	O5, O6	Camera capture, image compression, plate OCR (later)	MVP
Emergency SOS (6 issue types, GPS, cancel)	O2–O4	Live map, nearest-mechanic dispatch, background push, haptics	MVP
Mechanic found: name, phone, rating, ETA	O4	Live mechanic location on map, one-tap call	MVP
Book maintenance (4 services, date)	O7	Date strip, notes, calendar reminder	MVP
Diagnostics (symptoms + photo)	O8	Camera / short video, symptoms stored with job	MVP
Activity (bookings vs history)	O12	Offline cache of history	MVP
Live tracker + checklist	O9	Timeline, real-time via socket + push when backgrounded	MVP
Parts approval (approve/decline)	O10	Push → bottom sheet, total-impact preview	MVP
PDF receipt	O11	Open/share/save to device	MVP
Rating prompt	O11	Quick tags + comment; reminder push after 24 h	MVP
Owner profile	Profile	Notification settings, language (EN / Luganda later)	MVP
Mechanic duty toggle	M1	Background location while online (opt-in)	MVP
Mechanic job board (SOS/Bookings/Active/History)	M1	Distance-sorted, swipe actions	MVP
Global SOS pop-up	M2	Full-screen incoming alert, sound + vibration, lock-screen	MVP
Call owner / navigate	M3	In-app route preview, hand-off to Google Maps	MVP
Digital job card (reached car, checklist)	M4	Per-task photo evidence (fulfils FR-15)	MVP
Quote part (≤5 photos)	M5	Native camera, compression, upload retry	MVP
Finish job & receipt	M4	Lock until tasks done & quotes decided	MVP
Earnings (today / total)	M6	Weekly bar chart (fulfils FR-17)	MVP
Mechanic profile & reviews	Profile	Verification document upload	MVP

Feature (web today)	Screen	Native enhancement	Phase
Under-review screen	M7	Checklist of verification steps + push when approved	MVP
Admin console	—	Stays on the web (desktop use-case)	Web
Mobile-money payments	—	MTN MoMo / Airtel Money collection & payout	Phase 2
In-app chat	O4, O9	Owner ↔ mechanic messages per job	Phase 2
Offline SOS queue + SMS fallback	X1	Africa's Talking SMS short code	Phase 2
Service reminders	O5	Local notifications from last_service_date	Phase 2
AI-assisted diagnostics	O8	Symptom chat / OBD-II integration	Phase 3

4 Target architecture & technology stack

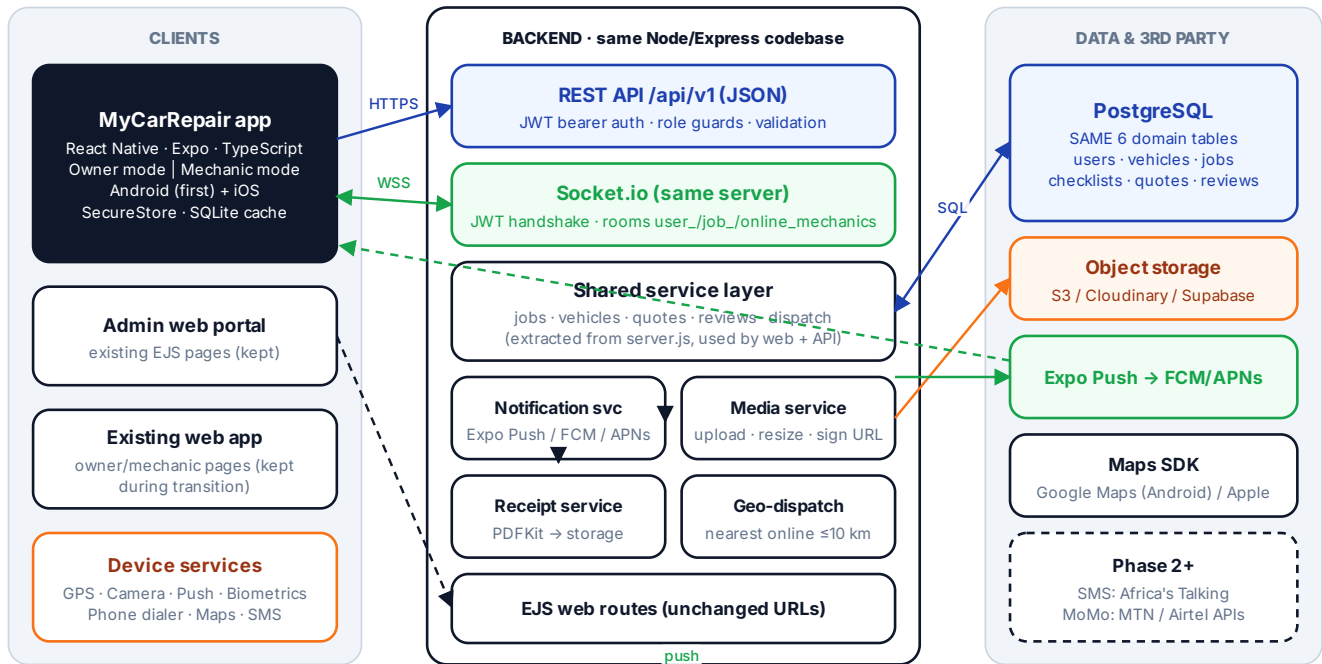


Figure 4.1 Target architecture: one backend and one database serving the web, the admin portal and the mobile app.

4.1 Key architectural points

- **Shared service layer.** Extract the SQL and rules from `server.js` into services (`jobService.accept()` , `quoteService.decide()` ...). Both the existing EJS routes and the new `/api/v1` controllers call them, so behaviour cannot drift.
- **Same Socket.io server.** Mobile clients authenticate with a JWT in the handshake (`auth.token`); web clients keep using the session cookie. The server decides which rooms a socket joins.
- **Push as a delivery channel for the same events.** When an event targets a user, the notification service also sends a push if that user has no active socket (app backgrounded or killed).
- **Media out of the web root.** Uploads go to object storage; the DB keeps the same comma-separated path columns, now holding storage keys or URLs.
- **Horizontal scaling ready.** Stateless JWT API plus the Socket.io Redis adapter lets the backend run multiple instances (NFR-09).

4.2 Technology stack

Concern	Choice	Why
Framework	React Native 0.7x + Expo SDK (managed workflow, dev client)	One JS/TS codebase for Android + iOS; same language as the Node backend; OTA updates
Language	TypeScript (strict)	Shared types with the API (models in Appendix A)
Navigation	Expo Router (file-based) / React Navigation	Auth stack + role-based tab navigators, deep links

Concern	Choice	Why
Server state	TanStack Query	Caching, retries, offline persistence, background refetch
Client state	Zustand	Session, role, online toggle, SOS state
Real-time	socket.io-client 4.x	Same protocol as the existing server
Forms	react-hook-form + zod	Validation mirrors server rules
Maps	react-native-maps (Google Maps SDK on Android)	SOS map, live mechanic tracking, route preview
Location	expo-location (+ expo-task-manager for background)	SOS fix, mechanic live location while online
Camera / media	expo-camera, expo-image-picker, expo-image-manipulator	Vehicle, diagnostic and part photos; resize to ≤1600 px, JPEG 0.7
Push	expo-notifications (FCM / APNs)	Job alerts, quotes, completion; full-screen SOS channel on Android
Secure storage	expo-secure-store, expo-local-authentication	Tokens in Keychain/Keystore, fingerprint/Face ID unlock
Files	expo-file-system, expo-sharing	Download & share PDF receipts
Charts	victory-native or react-native-gifted-charts	Earnings bar chart
Icons & fonts	lucide-react-native, Inter, Space Grotesk	Same visual language as the web
Quality	ESLint, Prettier, Jest, React Native Testing Library, Maestro (E2E)	
Build & release	EAS Build, EAS Submit, EAS Update	Play Store (internal → closed → production), TestFlight
Monitoring	Sentry (crashes), Expo Insights / PostHog (analytics)	
Backend additions	jsonwebtoken, zod, helmet, express-rate-limit, multer-s3 / Cloudinary SDK, expo-server-sdk	Added to the existing Express server

5 Data model — same models as the web

No changes to existing models

The mobile app uses the six domain tables exactly as they are in the web platform. The ERD below is the same one in the Web Platform Documentation. The API converts `snake_case` columns to `camelCase` JSON and splits comma-separated photo columns into arrays. The database is not altered.



Figure 5.1 Shared data model (web + mobile).

5.1 Column → API field mapping rules

Database	API / TypeScript	Rule
plate_number, is_online ...	plateNumber, isOnline ...	snake_case → camelCase
vehicles.photos TEXT	photos: string[]	split on ","; join on write; values become absolute (signed) URLs
parts_quotes.photo_evidence TEXT	photos: string[]	same as above
parts_quotes.is_approved NULL/true/false	isApproved: boolean null	null = awaiting owner (keep the tri-state)

Database	API / TypeScript	Rule
<code>DECIMAL(12,2)</code> prices	<code>number</code> (UGX, integer)	UGX has no minor unit in practice; format with thousands separators
<code>TIMESTAMP</code>	ISO-8601 string (EAT, UTC+3)	server returns UTC with offset
<code>password</code>	never returned	

5.2 How mobile features map onto existing columns

Mobile feature	Stored in (existing)
Mechanic live location / owner SOS location	<code>users.location_lat</code> , <code>users.location_lng</code>
Duty toggle, nearest-mechanic dispatch	<code>users.is_online</code> , <code>users.status</code> , lat/lng
Per-task photo evidence (FR-15)	<code>job_checklists.photo_url</code> (already in schema, unused by web)
Arrival confirmation code (optional)	<code>jobs.start_code</code> (already in schema)
Service-due reminder	<code>vehicles.last_service_date</code> (computed on device)
Review quick-tags	prefixed into <code>reviews.feedback</code> text
SOS issue / diagnostic symptoms	<code>jobs.service_type</code> (e.g. "Diagnostic: Engine Light, Overheating", ≤50 chars)
Maintenance mode / minimum app version	<code>system_config</code> key/value rows (new keys, same table)

5.3 The one storage addition: device push tokens

Push notifications need each device's Expo/FCM token. It must not go into the existing models. Choose one of these:

Option A (recommended) — auxiliary table

```
CREATE TABLE device_tokens (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON
DELETE CASCADE,
  token TEXT UNIQUE NOT NULL,
  platform VARCHAR(10),      -- android |
ios
  created_at TIMESTAMP DEFAULT now()
);
```

Purely additive and independent. Existing tables and web code are unaffected. Supports several devices per user.

Option B — no new table

Store tokens in `system_config` under key `push:<userId>` (JSON array). Zero migrations, but harder to query and clean up. Acceptable for a pilot.

Future phases (new tables, not MVP)

Phase 2: `messages` (chat), `payments` (MoMo transactions), `mechanic_documents` (verification uploads). These are new features, so they get new tables. The existing models still stay as they are.

6 Backend work: API v1, authentication, dispatch

Work required in the existing repository **before** the app can ship (Phase 0, ≈3 weeks). It also fixes the high-severity issues in the web documentation (§13).

#	Task	Details
1	Refactor into modules	<code>src/routes/web</code> , <code>src/routes/api/v1</code> , <code>src/services</code> , <code>src/sockets</code> , <code>src/middleware</code> . Keep every existing web URL working.
2	Migrations	Adopt the effective schema (Web doc, Appendix A) as migration 001; add <code>device_tokens</code> as 002.
3	JWT auth	<code>/auth/login register refresh logout</code> ; <code>requireAuth</code> , <code>requireRole('owner' 'mechanic')</code> , ownership guards on every <code>job/vehicle/quote/task</code> .
4	Socket auth & rooms	Accept <code>auth.token</code> (JWT) or session. Server joins <code>user_<id></code> ; <code>job_<id></code> for participants; drop client-chosen <code>join_room</code> . Replace every <code>io.emit</code> with targeted emits.
5	Geo-dispatch (FR-03)	Nearest ≤5 online mechanics within 10 km (Appendix SQL); widen to 20 km after 60 s with no acceptance.
6	Media service	<code>multer</code> → <code>memory</code> → <code>sharp</code> <code>resize</code> → <code>S3/Cloudinary</code> ; JPEG/PNG/WebP ≤5 MB; signed URLs; receipts stored privately.
7	Push service	<code>expo-server-sdk</code> ; notification matrix §8; Android channels <code>sos</code> (max priority) and <code>updates</code> .
8	Validation & errors	<code>zod</code> schemas, uniform error envelope, HTTP codes (400/401/403/404/409/422).
9	Security middleware	<code>helmet</code> , CORS allow-list, rate limit on auth and SOS, secrets from env, HTTPS only.
10	Config endpoint	<code>GET /api/v1/config</code> returns fee, issue/service/symptom lists, maintenance flag, <code>minAppVersion</code> for forced updates.
11	Tests	Supertest for every v1 endpoint, <code>socket.io</code> -client tests for dispatch and scoping (IT-01 ... IT-10).

6.1 Authentication flow

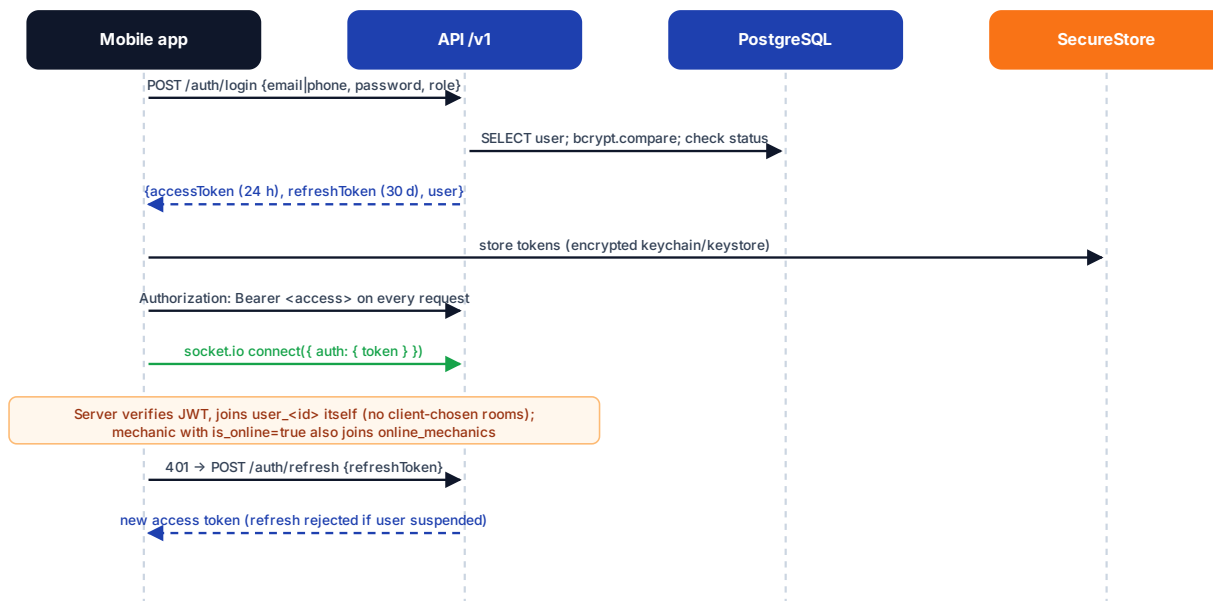


Figure 6.1 Mobile authentication and socket handshake.

6.2 Mobile SOS dispatch (target)

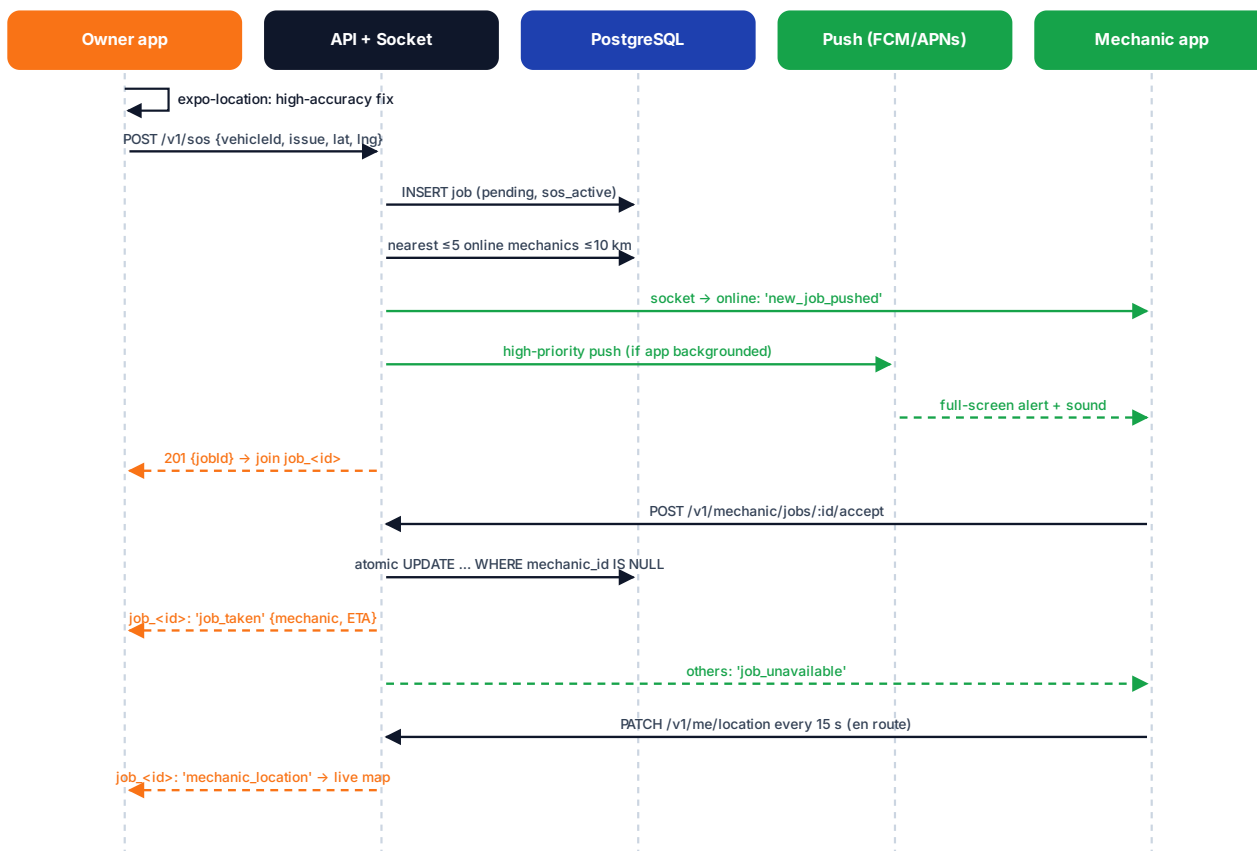


Figure 6.2 SOS with geo-dispatch, push to backgrounded mechanics and live location.

6.3 Nearest-mechanic query

```
-- FR-03: nearest 5 online, active mechanics within 10 km (uses existing columns only)
SELECT id, full_name, dist_km FROM (
  SELECT id, full_name,
    6371 * 2 * ASIN(SQRT(POWER(SIN(RADIANS(location_lat - $1) / 2), 2) +
      COS(RADIANS($1)) * COS(RADIANS(location_lat)) *
      POWER(SIN(RADIANS(location_lng - $2) / 2), 2))) AS dist_km
  FROM users
  WHERE role = 'mechanic' AND status = 'active' AND is_online
    AND location_lat IS NOT NULL
) m WHERE dist_km <= 10 ORDER BY dist_km LIMIT 5;
```

6.4 Behaviour fixes the app depends on

- SOS cancel must set `status='cancelled'` (web sets `completed`).
- Diagnostics must persist the chosen symptoms (web drops the `symptoms` field).
- Job completion must be refused while checklist items are open or quotes are still pending (web allows it).
- Owner-count of online mechanics must be numeric (web shows 0 because of a string comparison).

7 API contract (v1)

Base URL <https://api.mycarrepair.ug/api/v1> (example). JSON over HTTPS; **Authorization: Bearer <accessToken>**; list endpoints accept **page** & **limit** (default 20). All IDs are integers from the existing tables.

Auth & account

	Endpoint	Access	Request	Response / behaviour
POST	/auth/register	Public	{fullName, email, phone, password, role}	Owner → active; mechanic → pending. Returns tokens (owner) or 202 pending (mechanic)
POST	/auth/login	Public	{email phone, password, role}	{accessToken, refreshToken, user}; 403 suspended / pending
POST	/auth/refresh	Refresh token	{refreshToken}	New access token
POST	/auth/logout	Bearer	{pushToken?}	Invalidate device push token
GET	/me	Bearer	–	User profile (+ garage fields for mechanics)
PATCH	/me	Bearer	{fullName, phone, garageName?, garageLocation?, expertise?}	Updated user
POST	/me/location	Bearer	{lat, lng, accuracy?}	Writes users.location_lat/lng; relays to active job room if en route
POST	/me/push-token	Bearer	{token, platform}	Register device for push (see §5.3)
GET	/config	Public	–	Service fee, issue types, service types, symptoms, maintenance flag, min app version

Vehicles (owner)

	Endpoint	Access	Request	Response / behaviour
GET	/vehicles	Owner	–	Vehicle[] (photos as string[])
POST	/vehicles	Owner	multipart: fields + photos[≤5]	Vehicle
GET	/vehicles/:id	Owner (own)	–	Vehicle + recent jobs
PATCH	/vehicles/:id	Owner (own)	fields and/or new photos	Vehicle

Endpoint	Access	Request	Response / behaviour
DELETE /vehicles/:id	Owner (own)	–	204

Jobs (owner)

Endpoint	Access	Request	Response / behaviour
POST /jobs/bookings	Owner	{vehicleId, serviceType, scheduledDate, notes?}	Job (pending) + default checklist; dispatch to online mechanics
POST /jobs/diagnostics	Owner	multipart: {vehicleId, symptoms[], photo?}	Job (service_type "Diagnostic Scan")
POST /sos	Owner	{vehicleId, issue, lat, lng}	201 {jobId, nearbyCount, eta}; dispatch to nearest ≤5 online mechanics ≤10 km
POST /sos/:jobId/cancel	Owner (own)	–	status = cancelled
GET /jobs?scope=active history	Owner	page, limit	Job[] grouped client-side by date
GET /jobs/:id	Owner (own)	–	Job + mechanic + vehicle + checklist[] + quotes[] + totals
POST /quotes/:id/decision	Owner (job owner)	{decision: approve reject}	Quote; notifies mechanic
GET /jobs/:id/receipt	Owner / mechanic of job	–	Signed URL to PDF
POST /jobs/:id/review	Owner (own, completed)	{rating 1–5, feedback?, tags?}	Review (1 per job)

Mechanic

Endpoint	Access	Request	Response / behaviour
PATCH /mechanic/status	Mechanic	{isOnline}	Joins/leaves online_mechanics
GET /mechanic/jobs?tab=sos bookings active history	Mechanic	lat, lng	Job[] with distanceKm
GET /mechanic/stats	Mechanic	–	{totalJobs, activeJobs, rating}
POST /mechanic/jobs/:id/accept	Mechanic	–	Atomic claim; 409 if taken
POST /mechanic/jobs/:id/decline	Mechanic	–	Booking → cancelled (web parity)
POST /mechanic/jobs/:id/arrived	Assigned mechanic	–	status fixing; checklist activated

Endpoint	Access	Request	Response / behaviour
PATCH /mechanic/tasks/:taskId	Assigned mechanic	multipart: {isComplete, photo?}	Task + progress; emits task_update
POST /mechanic/jobs/:id/quotes	Assigned mechanic	multipart: {partName, price, photos[≤5]}	Quote (is_approved NULL)
POST /mechanic/jobs/:id/complete	Assigned mechanic	–	Totals, receipt, job_finished; 422 if tasks/quotes open
GET /mechanic/earnings?range=day week month	Mechanic	–	{total, series[], payouts[]}
GET /mechanic/reviews	Mechanic	page	Review[] + average

7.1 Example payloads

```
// POST /api/v1/sos
{ "vehicleId": 1, "issue": "Flat Tire", "lat": 0.3136, "lng": 32.5811 }
// 201 Created
{ "jobId": 42, "status": "pending", "nearbyCount": 3, "eta": "8 – 12 minutes" }

// GET /api/v1/jobs/1 (abridged)
{ "id": 1, "serviceType": "Brake Repair", "status": "fixing", "sosActive": false,
  "vehicle": { "make": "Toyota", "model": "Premio", "plateNumber": "UBK 482X" },
  "mechanic": { "id": 2, "fullName": "Joseph Okello", "phone": "+256 701 333444", "rating": 4.9 },
  "checklist": [ { "id": 1, "taskDescription": "Oil Level Checked", "isCompleted": true } ],
  "quotes": [ { "id": 2, "partName": "Brake Disc Rotor", "price": 210000, "isApproved": null,
    "photos": [ "https://cdn.../parts/part-1777453451335.jpeg" ] } ],
  "totals": { "serviceFee": 50000, "approvedParts": 120000, "total": 170000 } }

// Error envelope (all endpoints)
{ "error": { "code": "JOB_ALREADY_TAKEN", "message": "Another mechanic accepted this job." } } // HTTP
409
```

7.2 Web → API mapping

Existing web route	v1 equivalent
POST /auth/signup , /auth/login	/auth/register , /auth/login
POST /owner/add-car , /owner/car/:id/edit delete	POST /vehicles , PATCH DELETE /vehicles/:id
POST /owner/finalize-booking	POST /jobs/bookings , POST /jobs/diagnostics
socket emergency_sos / cancel_sos	POST /sos , POST /sos/:id/cancel (REST is reliable on flaky networks)
GET /owner/track-job/:id , /owner/job-details/:id	GET /jobs/:id
POST /api/owner/respond-quote , /approve-part	POST /quotes/:id/decision
POST /api/owner/rate-mechanic	POST /jobs/:id/review
POST /api/owner/update-location	POST /me/location

Existing web route	v1 equivalent
socket <code>accept_job</code> , <code>/api/mechanic/respond-appointment</code>	POST <code>/mechanic/jobs/:id/accept decline</code>
<code>/api/mechanic/reached-car</code> , <code>/verify-problem</code>	POST <code>/mechanic/jobs/:id/arrived</code>
<code>/api/mechanic/check-task</code>	PATCH <code>/mechanic/tasks/:taskId</code>
POST <code>/mechanic/job/:id/quote</code> , <code>/complete</code>	POST <code>/mechanic/jobs/:id/quotes</code> , <code>/complete</code>
<code>/api/mechanic/all-jobs</code> , <code>/stats</code> , <code>/toggle-status</code>	GET <code>/mechanic/jobs?tab=</code> , <code>/mechanic/stats</code> , PATCH <code>/mechanic/status</code>
GET <code>/mechanic/earnings</code> , <code>/mechanic/profile</code>	GET <code>/mechanic/earnings</code> , <code>/mechanic/reviews</code> , PATCH <code>/me</code>

8 Real-time events & push notifications

Event names are kept from the web platform so both clients can share the server. When the app is in the foreground it listens on the socket; when backgrounded or killed, the notification service sends a push that deep-links to the right screen.

Event	Room / target	Receiver	Push text (example)	Deep link
<code>new_job_pushed</code>	online_mechanics (nearest for SOS)	Mechanic	 New SOS: Dead Battery · 1.8 km	<code>mycarrepair://mechanic/job/:id (full-screen)</code>
<code>job_unavailable (new)</code>	mechanics who received the job	Mechanic	— (silent, removes card)	—
<code>job_taken</code>	job_<id> / user_<owner>	Owner	Joseph is on the way · 6 min	<code>mycarrepair://job/:id</code>
<code>mechanic_location (new)</code>	job_<id>	Owner	— (live map only)	—
<code>job_progress_update</code>	user_<owner>	Owner	Your mechanic has arrived / started	<code>mycarrepair://job/:id</code>
<code>task_update</code>	job_<id>	Owner	— (in-app only; digest push optional)	—
<code>new_quote_alert</code>	user_<owner>	Owner	Quote needs approval · UGX 210,000	<code>mycarrepair://quote/:id</code>
<code>quote_updated</code>	user_<mechanic>	Mechanic	Owner approved Brake Disc Rotor	<code>mycarrepair://mechanic/job/:id</code>
<code>appointment_update</code>	user_<owner>	Owner	Booking accepted by Okello Auto Works	<code>mycarrepair://job/:id</code>
<code>job_finished</code>	job_<id>	Owner	Job complete · receipt ready	<code>mycarrepair://job/:id/receipt</code>
<code>(scheduled) rating_reminder</code>	server job, +24 h	Owner	How was your repair? Rate Joseph	<code>mycarrepair://job/:id/review</code>
<code>(admin) mechanic_approved</code>	user_<mechanic>	Mechanic	You're verified — go online	<code>mycarrepair://mechanic</code>

Android notification channels

`sos` : max importance, custom siren sound, full-screen intent, bypasses DND where allowed.

`jobs` : high importance (quotes, arrivals, completion).

`general` : default (reminders, marketing: opt-in).

Reliability rules

Push payloads carry only IDs; the app re-fetches with `GET /jobs/:id`. Socket reconnects re-sync state. Every event is idempotent. Owners and mechanics never receive other users' events.

9 Information architecture & navigation

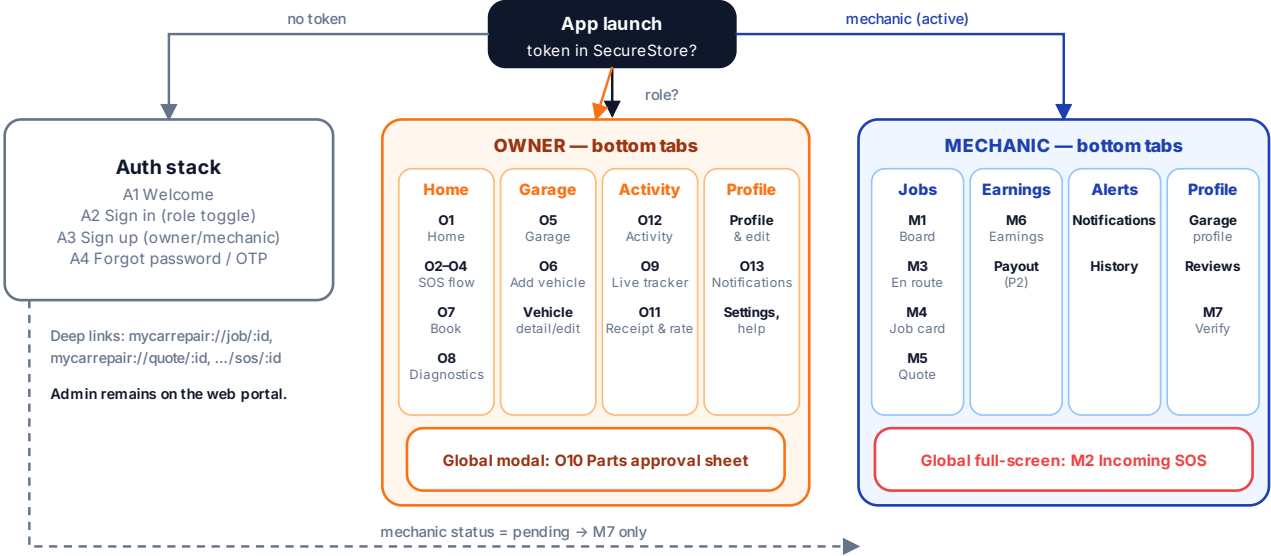


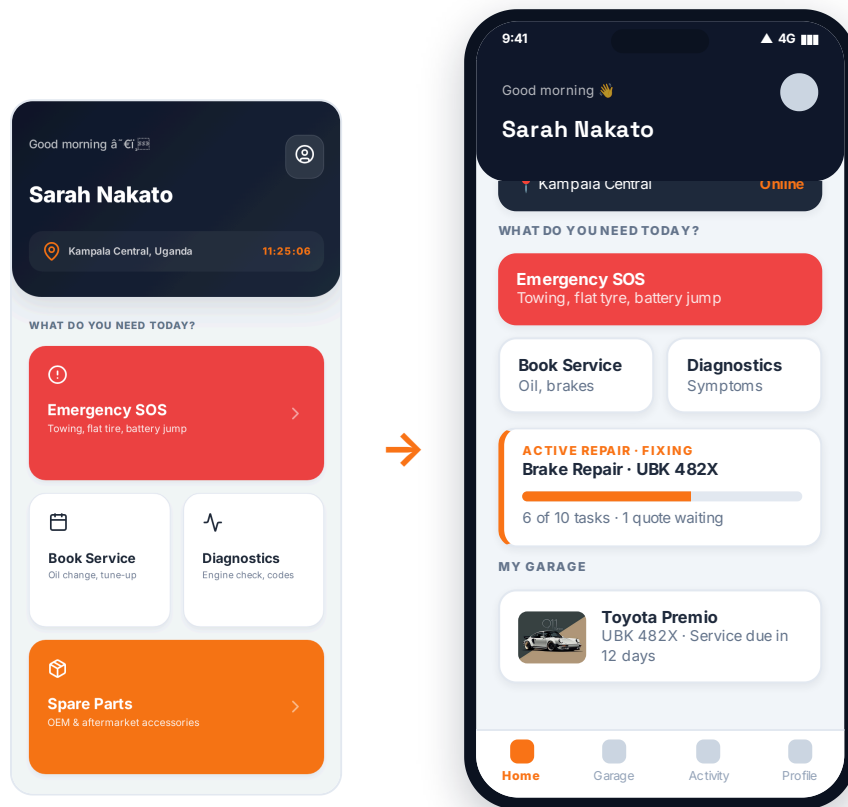
Figure 9.1 Navigation map: auth stack, role-based tab bars, global modals and deep links.

Rule	Detail
Launch routing	Valid token → role home; mechanic with status pending → M7; maintenance flag from <code>/config</code> → maintenance screen; outdated version → force-update screen.
Owner tabs	Home · Garage · Activity · Profile. SOS is on Home (big red card) and also as a long-press quick action on the app icon.
Mechanic tabs	Jobs · Earnings · Alerts · Profile. The Online toggle sits in the Jobs header.
Global overlays	Owner: parts-approval sheet can open from anywhere (push/deep link). Mechanic: incoming-SOS full screen interrupts any screen.
Back behaviour	Android hardware back mirrors the header back; SOS broadcasting asks for confirmation before leaving.
Deep links	Scheme <code>mycarrepair://</code> + universal links <code>https://mycarrepair.ug/...</code> : job, quote, receipt, review, mechanic job.

10 Screen specifications with mock-ups

Mock-ups are high-fidelity blueprints in the existing brand. Final visual design is refined in Figma during Phase 0. Vehicle and part photos are placeholders taken from the repository's uploads. Each group ends with a spec table for developers.

10.1 From web to app — key screens side by side

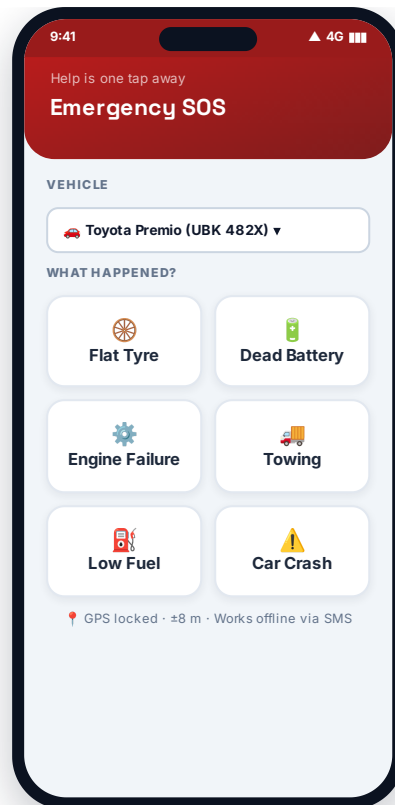
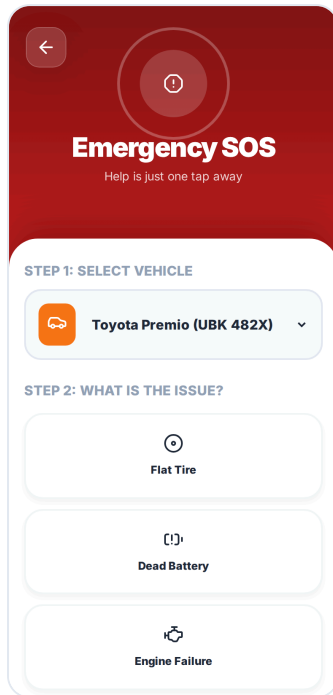


Owner dashboard → 01 Home

Same greeting header, service tiles and garage list. The app adds an **active repair card** with live progress and a pending-quote badge. The web bottom nav (Dashboard / Activity / Logout) becomes four tabs; logout moves to Profile.

01 · Owner home

SOS, services, active job, garage

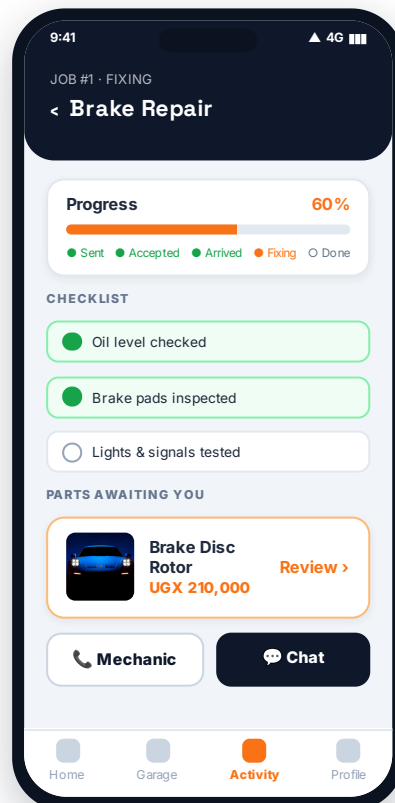
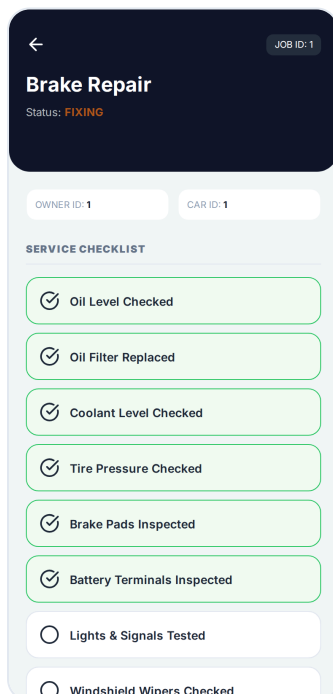


Emergency SOS → 02

The same six issue types in a 2-column grid of large targets. The vehicle is preselected. The GPS lock and accuracy are shown before sending. The request goes over REST ([POST /sos](#)) so it survives socket reconnects.

02 · SOS — choose issue

One tap sends GPS + vehicle

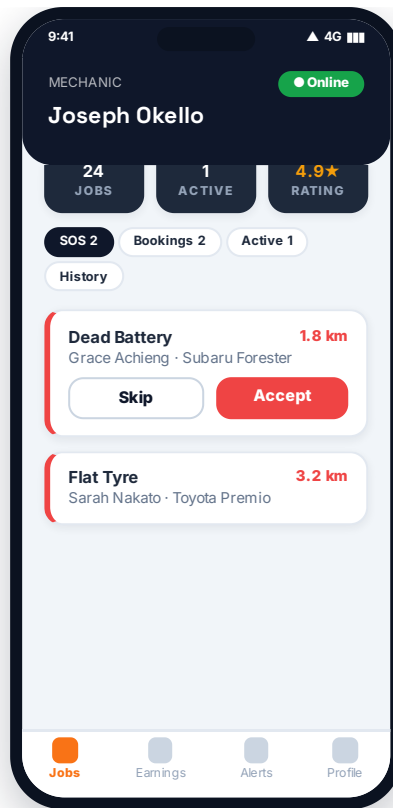
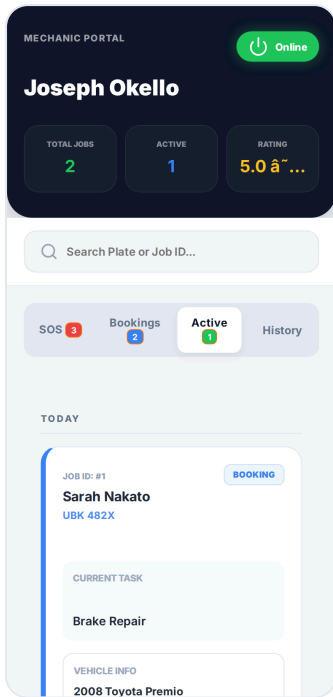


Live tracker → 09

The checklist and quotes from the web become a **5-stage timeline** plus a compact checklist. Pending quotes appear as a card that opens the approval sheet (O10). Call and chat are one tap away.

09 · Live repair tracker

5-stage timeline, live checklist

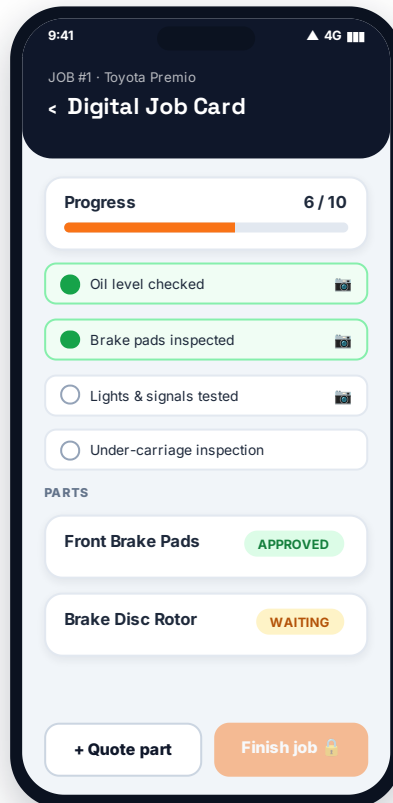
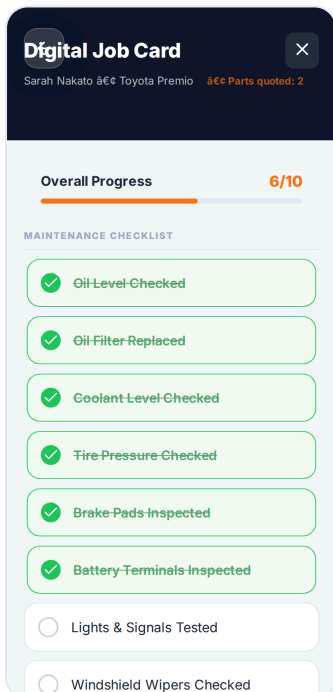


M1 · Job board

Duty toggle, tabs, distance-sorted

Mechanic board → M1

Same stats row and SOS / Bookings / Active / History tabs. Cards show **distance** and inline Accept / Skip. The Online toggle also starts or stops background location sharing.

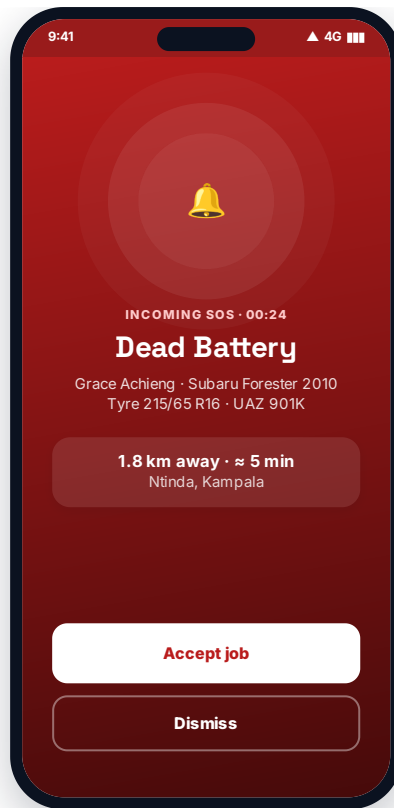
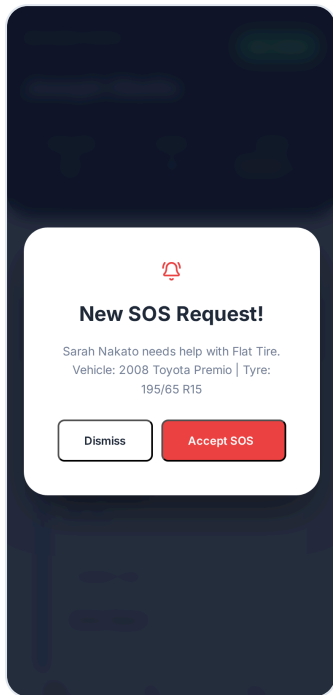


M4 · Digital job card

Tap tasks, photo proof, lock rules

Digital job card → M4

Same checklist and part statuses. Each task can carry a **photo proof** (fulfils FR-15 via [job_checklists.photo_url](#)). "Finish job" stays locked until all tasks are done and all quotes are decided.



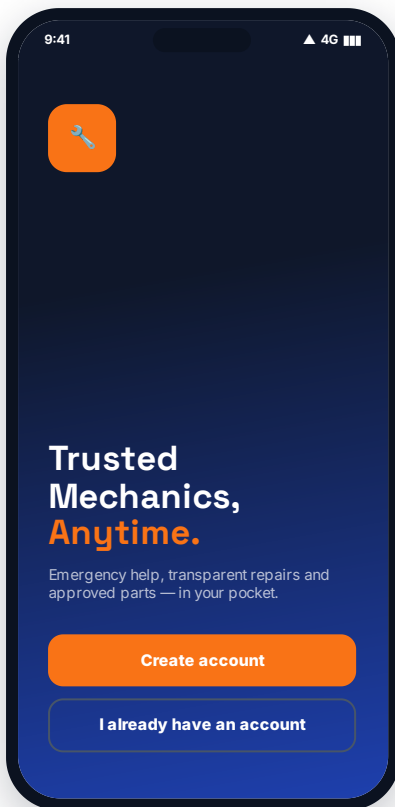
M2 · Incoming SOS

Full-screen, sound + vibration

SOS pop-up → M2 Incoming SOS

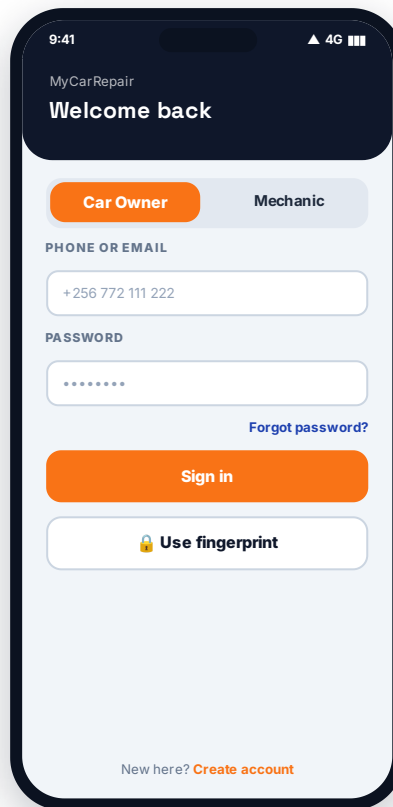
The web modal only works while the page is open. The app uses a **high-priority push with a full-screen intent**, a siren sound and vibration, so the mechanic sees it even when the phone is locked.

10.2 Onboarding & authentication



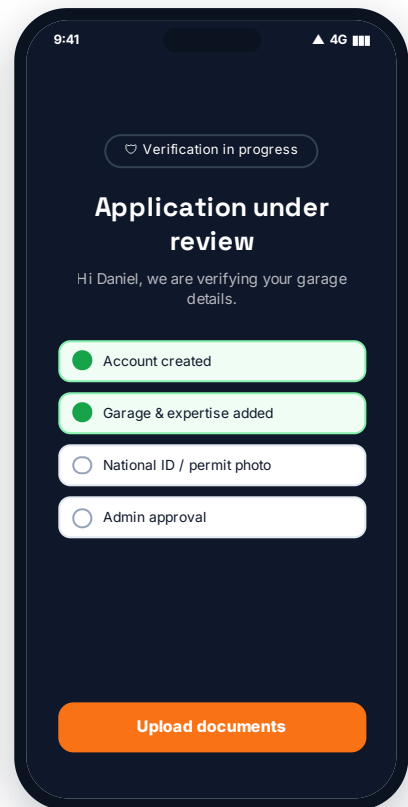
A1 · Welcome

First launch, language, sign-in



A2 · Sign in

Role toggle, biometrics, JWT

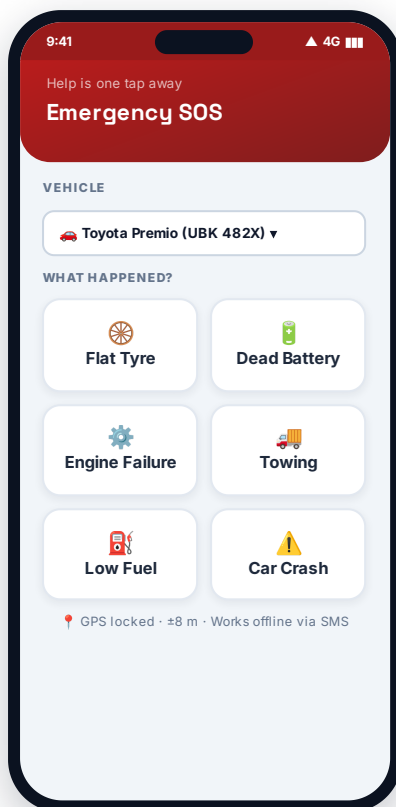


M7 · Verification status

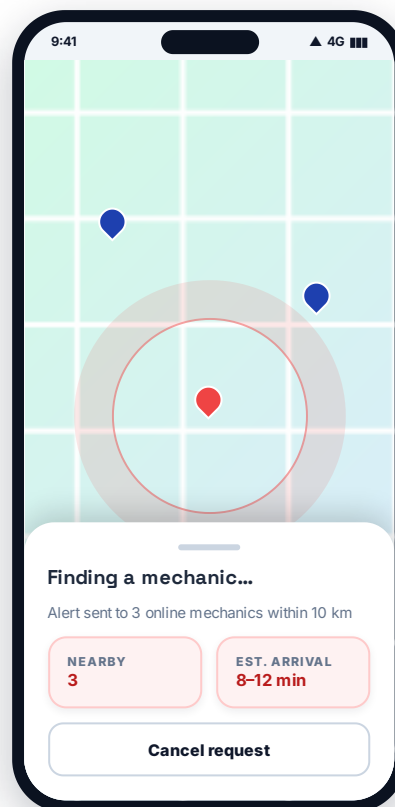
Pending mechanics (status=pending)

Screen	Purpose	API	Real-time / push	States & rules
A1 Welcome	Explain value; choose sign-up or sign-in.	—	—	Shown once; language picker (EN default).
A2 Sign in	Role toggle like the web; email or phone + password.	<code>POST /auth/login</code>	—	Biometric unlock after first login; 403 messages for suspended / pending mirror the web.
A3 Sign up	Name, email, phone (+256), password, role; mechanics add garage, location, expertise.	<code>POST /auth/register</code>	—	Mechanics land on M7 (pending).
M7 Verification	Holding screen for pending mechanics.	<code>GET /me</code>	push <code>mechanic_approved</code>	Document upload is Phase 2 (needs <code>mechanic_documents</code>); MVP shows Contact Admin.

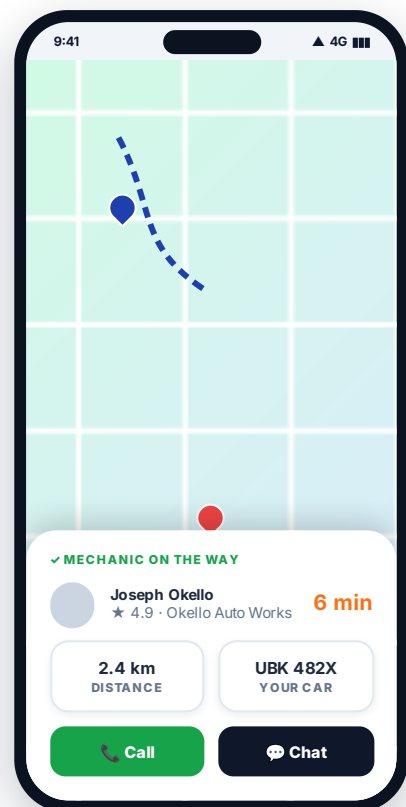
10.3 Car owner — emergency flow



O2 · SOS — choose issue
One tap sends GPS + vehicle



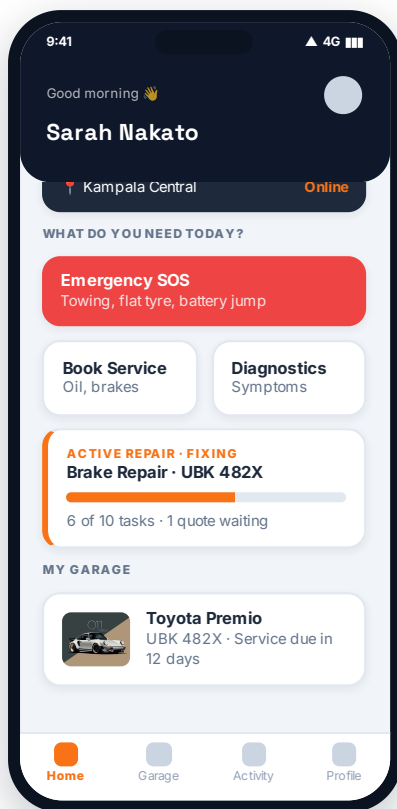
O3 · SOS — broadcasting
Live map of nearby online mechanics



O4 · Mechanic found
Live location, ETA, call / chat

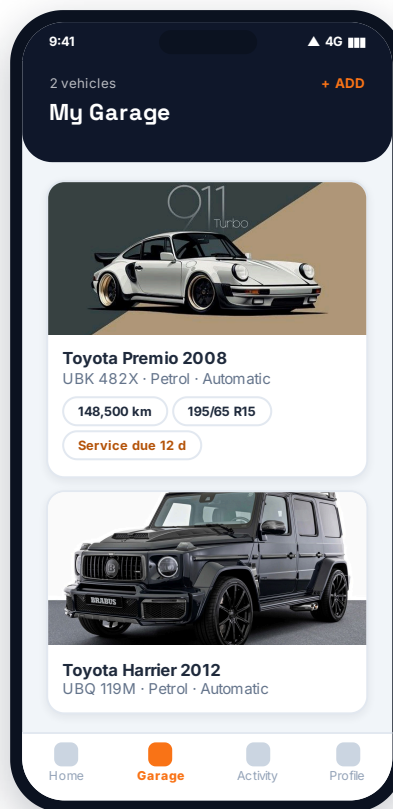
Screen	Purpose	API	Real-time / push	States & rules
O2 SOS — issue	Choose vehicle + 1 of 6 issues; tap sends immediately.	<code>POST /v1/sos</code>	—	Requires GPS fix (show accuracy). No vehicle → quick-add. Offline → X1 queue.
O3 SOS — broadcasting	Reassure the user while mechanics are alerted.	Response of <code>POST /sos</code> (nearbyCount, eta)	job_taken → O4; timeout 3 min → "widen search" / call support	Cancel → <code>POST /sos/:id/cancel</code> (status cancelled , not completed).
O4 Mechanic found	Show who is coming, live position and ETA.	<code>GET /jobs/:id</code>	mechanic_location (15 s), job_progress_update (arrived)	Call via dialer; Chat is Phase 2 (hide in MVP). Keep screen awake.

10.4 Car owner — home, garage & services



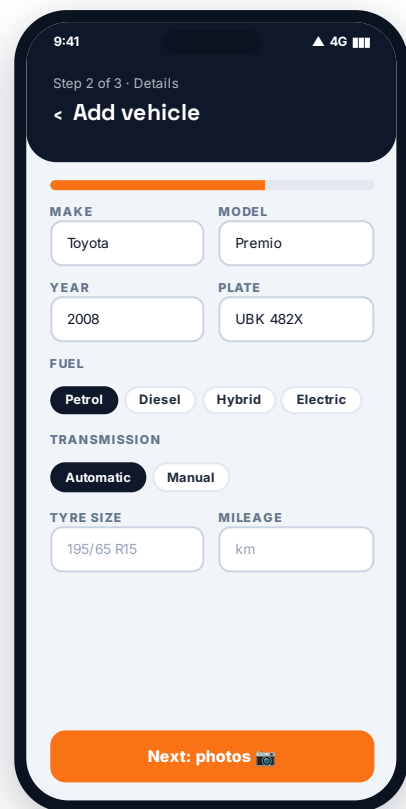
O1 · Owner home

SOS, services, active job, garage



O5 · Virtual garage

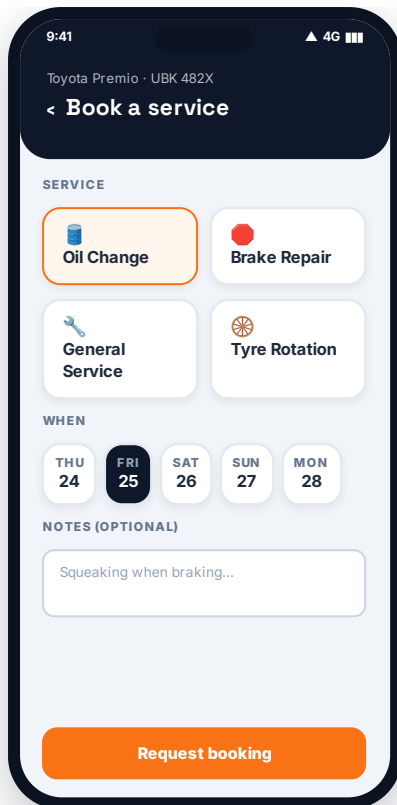
Cards with photos & reminders



O6 · Add vehicle wizard

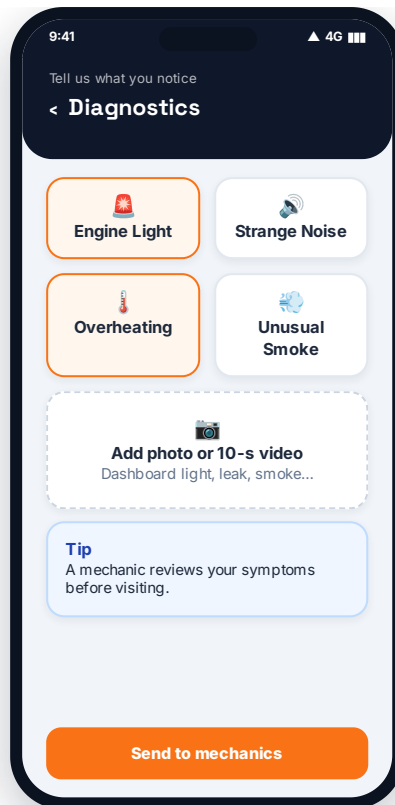
Same 11 fields, 3 short steps

Screen	Purpose	API	Real-time / push	States & rules
O1 Home	Entry point for owners; fastest path to SOS (≤ 1 tap).	<code>GET /me</code> , <code>GET /vehicles</code> , <code>GET /jobs?scope=active</code>	<code>job_progress_update</code> , <code>appointment_update</code> , <code>job_finished</code> → refresh card	Empty garage → "Add your first car" CTA. Location permission prompt explains SOS use.
O5 Garage	List vehicles with photo, key specs and service due.	<code>GET /vehicles</code>	—	Service due = <code>last_service_date</code> + 6 months (client-side; no schema change).
O6 Add vehicle	Same 11 fields as web, split in 3 steps (basics, specs, photos).	<code>POST /vehicles</code> multipart	—	Plate format hint (UAx/UBx 000X), photos compressed before upload, max 5.



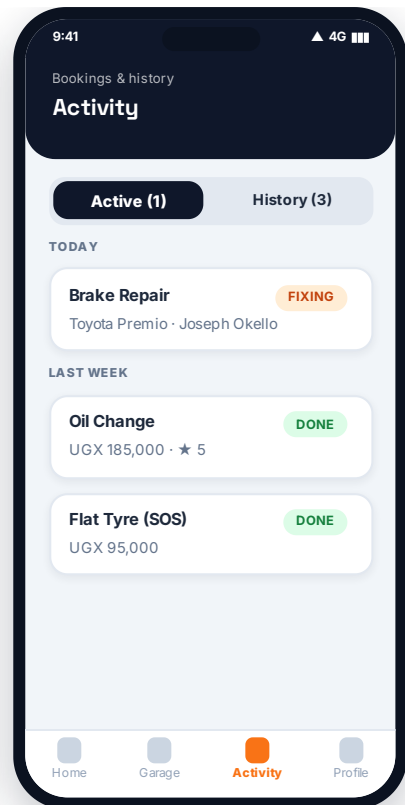
O7 · Book maintenance

Service, date strip, notes



O8 · Diagnostics

Symptoms + camera evidence

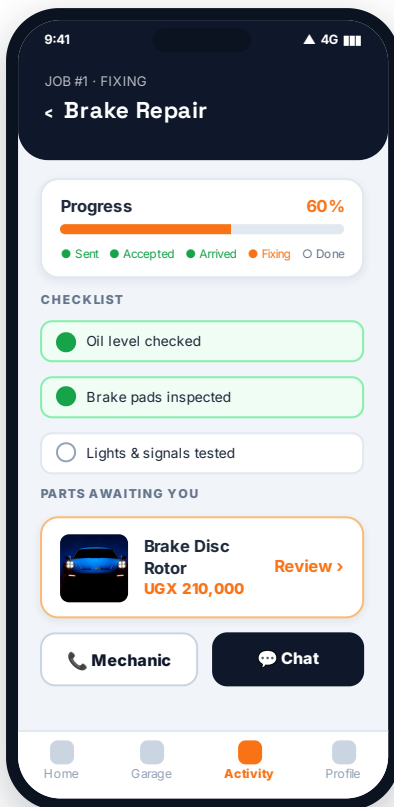


O12 · Activity

Active vs history, date groups

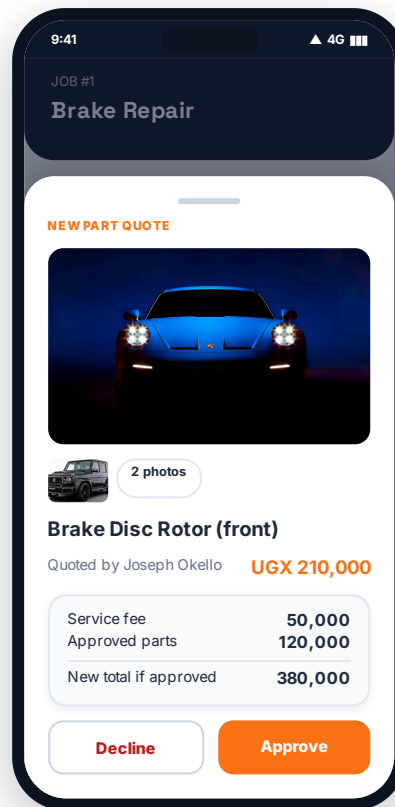
Screen	Purpose	API	Real-time / push	States & rules
O7 Book	Scheduled service with date strip.	<code>POST /jobs/bookings</code>	appointment_update	Past dates disabled (server also rejects).
O8 Diagnostics	Symptoms + photo/video evidence.	<code>POST /jobs/diagnostic</code> <code>s</code>	appointment_update	Web currently ignores the selected symptoms — API must persist them (see §6.4).
O12 Activity	Active vs history grouped by date.	<code>GET /jobs?scope=</code>	—	Cached for offline viewing. Same grouping as web (Today ... Older).

10.5 Car owner — tracking, approval, completion



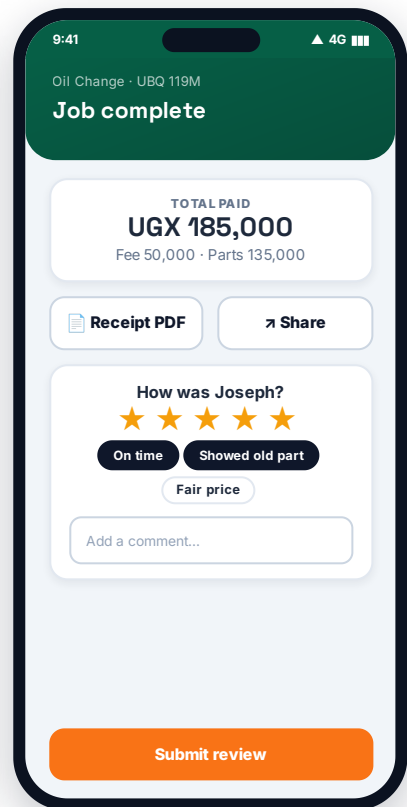
O9 · Live repair tracker

5-stage timeline, live checklist



O10 · Parts approval

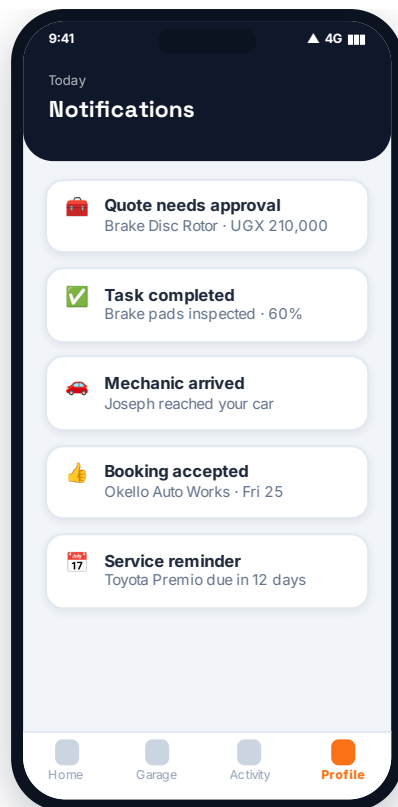
Push → sheet with photos & total impact



O11 · Receipt & rating

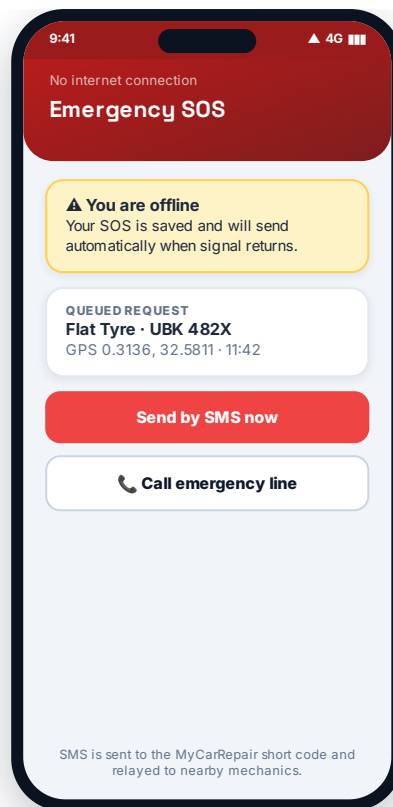
PDF, share, 1–5 stars + tags

Screen	Purpose	API	Real-time / push	States & rules
O9 Live tracker	Transparent progress: 5-stage timeline, checklist, pending quotes.	GET /jobs/:id	task_update, job_progress_update, new_quote_alert, job_finished	Stages map from status: pending→Sent, accepted→Accepted, fixing(first task)→Arrived/Fixing, completed→Done.
O10 Parts approval	Core trust feature: decide before purchase.	POST /quotes/:id/decision	Opened from push/deep link or tracker	Shows new total if approved (50,000 fee + approved parts). Decision is final (confirm dialog).
O11 Receipt & rating	Close the loop: pay summary, PDF, review.	GET /jobs/:id/receipt , POST /jobs/:id/review	job_finished	One review per job; tags stored inside feedback text (no schema change).



O13 · Notification centre

Push history with deep links

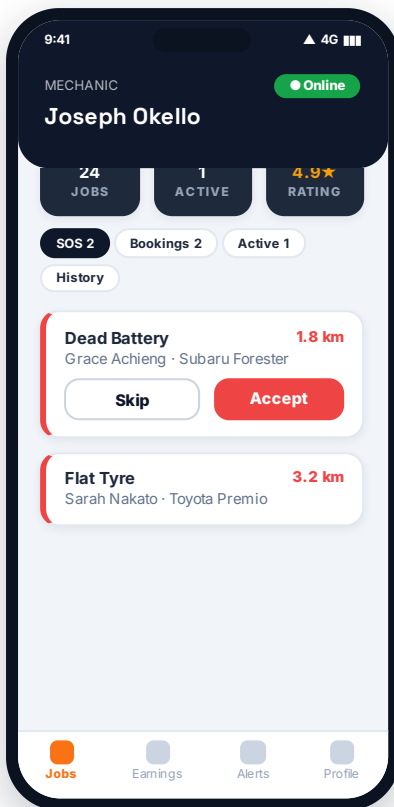


X1 · Offline SOS

Queue + SMS fallback (P2)

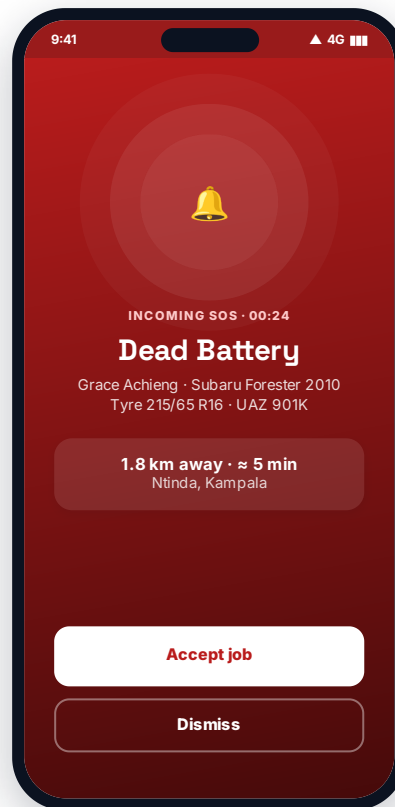
Screen	Purpose	API	Real-time / push	States & rules
O13 Notifications	History of pushes; tap = deep link.	Local store + <code>GET /job s/:id</code>	all owner events	Badge on Profile tab; mark-all-read.
X1 Offline SOS	Never lose an emergency request.	Queued <code>POST /sos</code>	—	Phase 2 SMS relay; MVP: queue + auto-retry + call button.

10.6 Mechanic



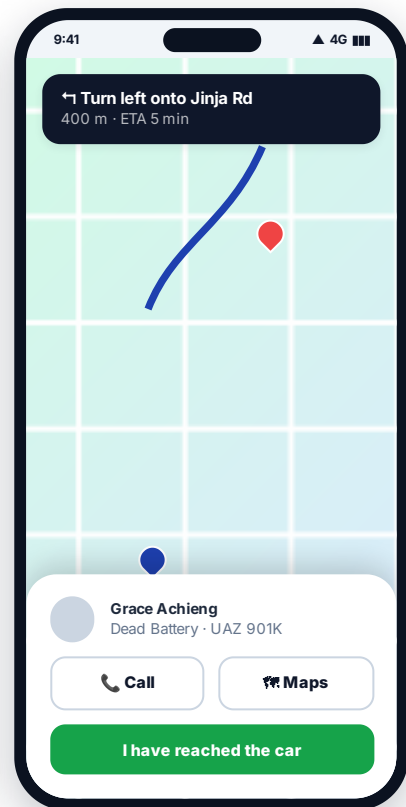
M1 · Job board

Duty toggle, tabs, distance-sorted



M2 · Incoming SOS

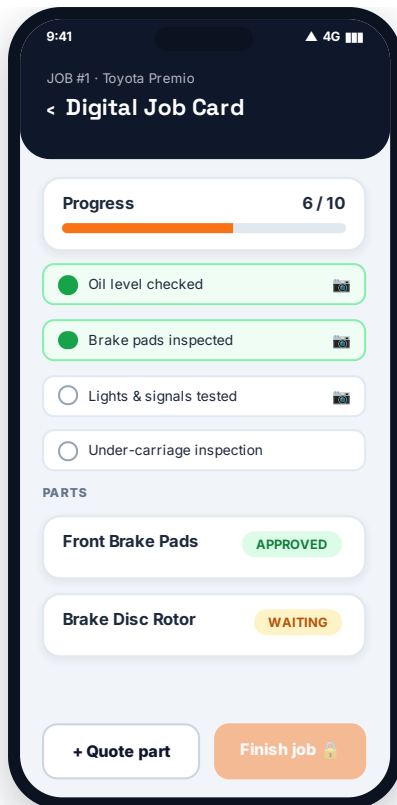
Full-screen, sound + vibration



M3 · En route

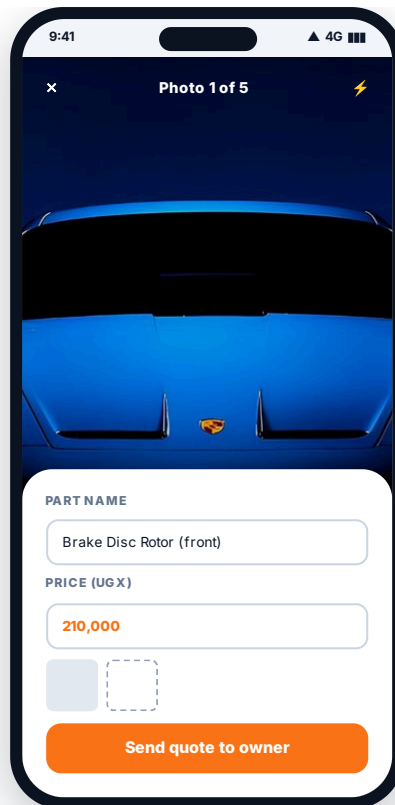
Sharing live location, arrive

Screen	Purpose	API	Real-time / push	States & rules
M1 Job board	Duty toggle and all work in one place.	GET /mechanic/jobs?tab= , /mechanic/stats	<code>new_job_pushed</code> , <code>job_unavailable</code>	Offline mode hides SOS tab and stops location. Pull-to-refresh.
M2 Incoming SOS	Win the job fast; full-screen even on lock screen.	POST /mechanic/jobs/:id/accept	push (high priority, own channel with siren sound)	30-s countdown; "Too late" state on 409.
M3 En route	Navigate and share live location with the owner.	POST /me/location every 15 s, /arrived	emits <code>mechanic_location</code>	Background location only while a job is accepted. Hand-off to Google Maps.



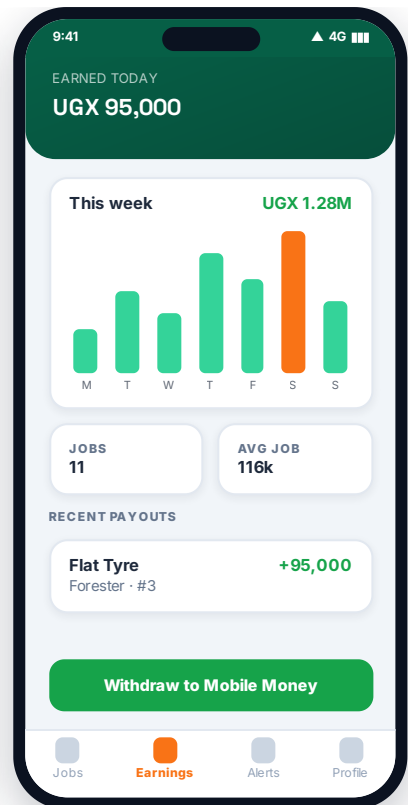
M4 · Digital job card

Tap tasks, photo proof, lock rules



M5 · Quote with camera

Capture, compress, send



M6 · Earnings

Daily/weekly chart, MoMo payout (P2)

Screen	Purpose	API	Real-time / push	States & rules
M4 Job card	Execute and document the repair.	PATCH /mechanic/tasks/:id , /complete	quote_updated	"Finish" locked until every task is ticked and no quote is pending. Optional photo per task → job_checklists.photo_url .
M5 Quote	Photo-first part quote.	POST /mechanic/jobs/:id/quotes	→ owner push	Up to 5 photos, retries on weak network, price in UGX with thousands separators.
M6 Earnings	Motivation + transparency on income.	GET /mechanic/earnings	job_finished	Daily bars for the week; MoMo withdrawal is Phase 2 (button hidden in MVP).

Common UI states (all screens)

State	Treatment
Loading	Skeleton cards (never blank screens); SOS shows radar animation.
Empty	Friendly illustration + one primary action (e.g. "Add your first car").
Error	Inline message + Retry; network errors show an offline banner; 409 conflicts get a specific message.
Offline	Cached data read-only, actions queued where safe (SOS, task ticks), banner at top.
Permissions denied	Explain why (e.g. GPS for SOS) + button to open settings; SOS falls back to manual location text.

11 Native capabilities, permissions & offline

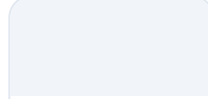
Capability	Who	Used for	Permission & rationale text
Location (foreground)	Owner, mechanic	SOS position, distance-sorted jobs	"MyCarRepair uses your location to send help to exactly where you are."
Location (background)	Mechanic	Live position to owner while a job is accepted	Only while Online + active job; persistent notification "Sharing location with Sarah".
Camera / photos	Both	Vehicle, diagnostic, part and task photos	Asked on first photo action, not at launch.
Notifications	Both	Job alerts, quotes, completion	Asked after sign-in with a pre-prompt explaining SOS alerts.
Full-screen intent	Mechanic (Android)	Incoming SOS on lock screen	Declared in manifest; Android 14 user toggle checked.
Phone dialer	Both	Call the other party	No permission (uses <code>tel:</code> link).
Biometrics	Both	Quick unlock	Optional, from Settings.
Maps hand-off	Mechanic	Turn-by-turn in Google Maps	Deep link <code>google.navigation:q=lat,lng</code> .

11.1 Offline & low-bandwidth design

- **Data-light:** images resized on device (≤ 1600 px, ~200 KB); list endpoints paginated; gzip; no auto-playing media.
- **Cache:** TanStack Query persisted to storage — garage, activity and job details open offline.
- **Queue:** SOS and checklist ticks are queued and replayed in order when back online (idempotency keys prevent duplicates).
- **Socket resilience:** exponential reconnect; on reconnect the app re-fetches the active job.
- **Phase 2:** SMS SOS via Africa's Talking short code when there is no data connection (dissertation §5.5.1).

12 Design system for mobile

The app keeps the web brand and the **Grease-Proof UI** principle: large targets, high contrast, one primary action per screen.

					
navy / surface #0f172a	blue / info #1e40af	orange / action #f97316	red / SOS only #ef4444	green / success #16a34a	canvas #f1f5f9

Token / rule	Value
Touch targets	≥ 48 × 48 dp (Android guidance; stricter than the web's 44 px); primary buttons 56 dp tall
Spacing scale	4 · 8 · 12 · 16 · 24 · 32 dp; screen padding 16 dp
Radius	cards 16 dp, buttons 14 dp, sheets 24 dp
Typography	Inter 14/16 body, 12 caption, 11 uppercase labels (+0.08 em); Space Grotesk 22–32 headings; supports system font scaling up to 130 %
Contrast	WCAG AA minimum; outdoor mode = extra-bold weights on dark headers
Colour use	Red is reserved for SOS/emergency; orange for the primary action; green for success/money
Icons	lucide-react-native, 24 dp, 2 px stroke; emoji avoided in UI text (fixes web mojibake)
Motion	200–300 ms; spring for sheets; SOS radar pulse; haptic feedback on SOS send, accept and approve
Dark mode	Supported from MVP (navy surfaces already in the palette)
Components	Button, IconButton, Card, ServiceTile, StatusPill, ChecklistRow, QuoteCard, PhotoPicker, MapCard, BottomSheet, Toast, EmptyState, Skeleton

13 Security, privacy & non-functional requirements

13.1 Security

- TLS everywhere; certificate pinning considered for Phase 2.
- Tokens only in SecureStore (Keychain/Keystore) — never AsyncStorage; refresh token rotation; logout revokes the device push token.
- Server-side authorisation on **every** resource (role + ownership). The app hides UI but never relies on hiding.
- Phone numbers and exact locations are revealed only to the counter-party of an accepted job, and location sharing stops when the job completes.
- Rate limits: login 5/min/IP, SOS 3/10 min/user; uploads virus-scanned or type-sniffed.
- No secrets in the app bundle (only public keys such as the Maps key restricted by package/SHA).

13.2 Privacy & compliance

Comply with Uganda's **Data Protection and Privacy Act, 2019** (registration with the Personal Data Protection Office, consent, purpose limitation, data subject rights) and with Google Play / App Store data-safety declarations. Provide an in-app privacy policy, account deletion (required by both stores) and data export on request.

13.3 Non-functional requirements (adapted from the dissertation)

Ref	Category	Requirement for the mobile app
NFR-01	Performance	Cold start ≤ 3 s on a 2 GB Android; screens interactive ≤ 1.5 s on 4G
NFR-02	Performance	SOS reaches targeted mechanics ≤ 2 s (socket) / ≤ 5 s (push)
NFR-03	Performance	Tracker updates ≤ 1 s after a task tick
NFR-04	Security	JWT (24 h) + refresh; bcrypt; HTTPS; secure storage
NFR-05	Security	Role-based access enforced by the API
NFR-06	Availability	99 % uptime 07:00–22:00 EAT; graceful maintenance screen
NFR-07	Usability	≥ 48 dp targets; primary actions ≤ 2 taps; works from 360 dp width; font scaling
NFR-08	Compatibility	Android 8.0+ (API 26), iOS 15+
NFR-09	Scalability	Stateless API, Socket.io Redis adapter, CDN for media
NFR-10	Size & data	Android download ≤ 35 MB; ≤ 5 MB data for a typical repair flow
NFR-11	Reliability	Crash-free sessions ≥ 99.5 % (Sentry)
NFR-12	Accessibility	Screen-reader labels on all controls; colour never the only signal

14 Project structure, testing, CI/CD & release

14.1 Mobile repository layout

```
mycarrepair-mobile/
├── app/                                # Expo Router screens
│   ├── (auth)/                        # welcome, sign-in, sign-up, forgot-password
│   ├── (owner)/                      # tabs: home, garage, activity, profile
│   │   ├── sos/                      # issue, broadcasting, found
│   │   │   ├── job/[id].tsx          # live tracker, receipt, review
│   │   │   └── quote/[id].tsx        # parts approval sheet
│   │   ├── (mechanic)/               # tabs: jobs, earnings, alerts, profile
│   │   │   ├── incoming/[id].tsx     # full-screen SOS
│   │   │   └── job/[id]/             # en-route, card, quote
│   │   └── _layout.tsx               # auth gate + role routing + push/deep-link handlers
├── src/
│   ├── api/                          # axios/fetch client, endpoints, zod schemas
│   ├── models/                      # TypeScript types (Appendix A) – same as DB
│   ├── realtime/                    # socket.io client, event hooks
│   ├── features/                    # sos, garage, jobs, quotes, earnings, auth
│   ├── components/                  # Button, Card, StatusPill, ChecklistRow, PhotoPicker, MapView...
│   ├── theme/                       # tokens: colours, spacing, typography
│   ├── services/                    # location, camera, notifications, storage
│   └── i18n/                        # en (Luganda / Swahili later)
├── assets/                          # icon, splash, sounds (sos_siren.wav)
├── app.config.ts                    # permissions, deep-link scheme, EAS project
└── eas.json                         # build profiles: development, preview, production
```

14.2 Testing strategy

Level	Tools	Scope
Unit	Jest	Formatters (UGX), status → stage mapping, validators, stores
Component	React Native Testing Library	ChecklistRow, QuoteCard, forms, empty/error states
API contract	zod schemas shared with backend; Supertest	Every v1 endpoint, auth and ownership (negative tests)
Real-time	socket.io-client test harness	IT-02/03/04/10: dispatch, first-come, scoping, no cross-contamination
E2E	Maestro on Android emulator + one low-end device	SOS happy path, booking, quote approval, mechanic job to completion
UAT	Kampala pilot: 3 owners + 2 mechanics (dissertation §4.10)	Ease of use, clarity, likelihood of use (5-point scale)

14.3 CI/CD & release

- **CI (GitHub Actions):** lint, type-check, unit tests on every PR; backend tests against a Postgres service container.
- **Builds:** EAS profiles `development` (dev client), `preview` (internal APK for testers), `production` (AAB / IPA).

- **Distribution:** Play Console internal testing → closed testing (pilot) → production staged rollout 10 % → 100 %; TestFlight for iOS.
- **OTA:** EAS Update for JS-only fixes; `minAppVersion` in `/config` forces native updates.
- **Environments:** dev / staging / production API URLs and Maps keys via `app.config.ts` + EAS secrets.

15 Roadmap, team & definition of done

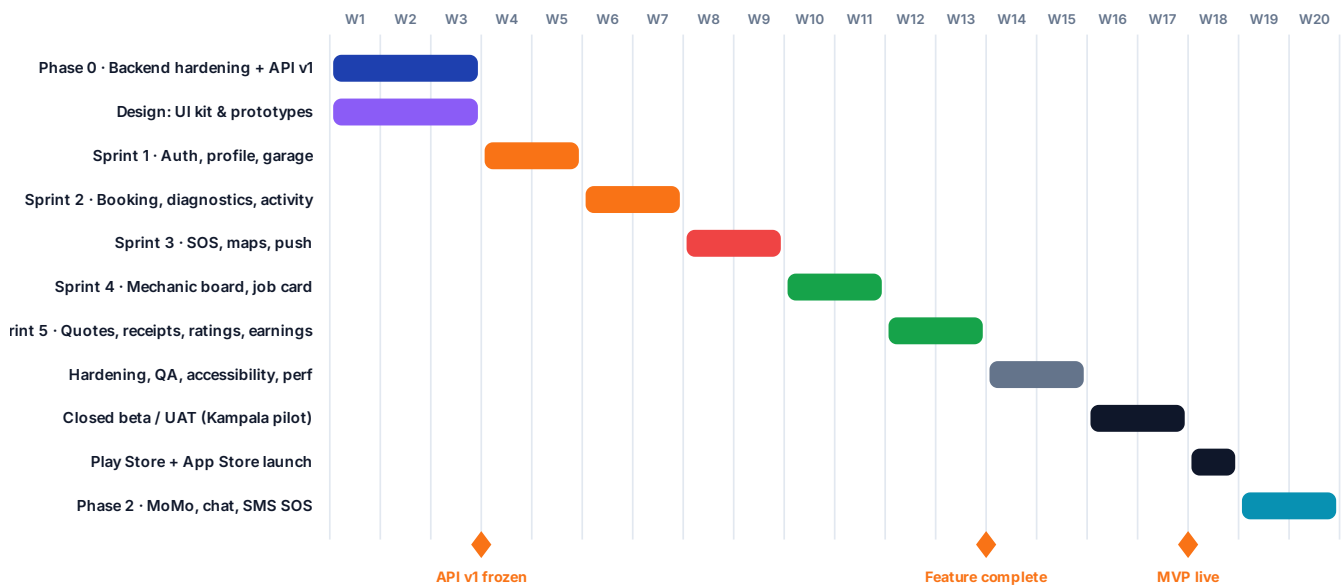


Figure 15.1 Indicative 20-week plan (2-week sprints). Durations assume the team below.

15.1 Suggested team

Role	Allocation
Product owner (Ian)	Priorities, acceptance, pilot recruitment
Backend developer	1 FTE — Phase 0 API, push, dispatch
React Native developers	1–2 FTE
UI/UX designer	0.5 FTE in weeks 1–6, then on call
QA engineer	0.5 FTE from week 7
Ops / admin	Mechanic verification during pilot

15.2 Milestones

Milestone	Exit criteria
M1 - API v1 frozen (W3)	All §7 endpoints live on staging with tests; HIGH web issues fixed
M2 - Feature complete (W13)	All MVP rows of §3 implemented
M3 - Pilot (W15–16)	UAT with Kampala owners & mechanics; crash-free ≥ 99 %
M4 - Launch (W18)	Play Store production, TestFlight/App Store submitted
M5 - Phase 2 (W19+)	MoMo payments, chat, SMS SOS

15.3 Definition of done (per feature)

- Matches the spec in §10 and handles loading / empty / error / offline states.
- API endpoints validated, authorised and covered by tests; events scoped to the right room.
- Works on a 2 GB Android device on a throttled 3G profile; screen-reader labels present.
- Analytics events and Sentry breadcrumbs added; copy reviewed; reviewed PR merged.

15.4 Open questions for the product owner

Question	Default if not decided
Is the UGX 50,000 service fee fixed for every job, or does it vary by service type / SOS?	Keep fixed, read from /config
Do owners pick a mechanic for bookings (FR-05) or keep auto-dispatch?	Keep auto-dispatch (web behaviour)
Platform commission on jobs?	None in MVP
Mechanic verification documents required?	National ID + trading licence (Phase 2 upload)
Brand domain / API host?	mycarrepair.ug (placeholder)
Keep the owner/mechanic web pages after launch?	Keep for 6 months, then redirect to app stores

A Appendix — TypeScript models, payloads, SQL

A.1 Shared TypeScript models (same as the database)

```
// Mirrors the existing PostgreSQL tables 1:1 (camelCase over the wire).
export type Role = 'owner' | 'mechanic' | 'admin';
export type UserStatus = 'active' | 'pending' | 'suspended';
export type JobStatus = 'pending' | 'accepted' | 'diagnosing' | 'fixing' | 'ready' | 'completed' | 'cancelled';

export interface User { // users
  id: number; fullName: string; email: string; phone: string | null;
  role: Role; status: UserStatus; isOnline: boolean;
  locationLat: number | null; locationLng: number | null;
  garageName?: string | null; garageLocation?: string | null; expertise?: string | null;
  createdAt: string;
}
export interface Vehicle { // vehicles
  id: number; ownerId: number; make: string; model: string; year: number;
  plateNumber: string; fuelType: 'Petrol' | 'Diesel' | 'Hybrid' | 'Electric';
  transmission: 'Automatic' | 'Manual'; tyreSize: string | null; color: string | null;
  mileage: number | null; lastServiceDate: string | null;
  photos: string[]; // API splits the comma-separated "photos" column
  createdAt: string;
}
export interface Job { // jobs
  id: number; ownerId: number; mechanicId: number | null; vehicleId: number;
  serviceType: string; status: JobStatus; totalPrice: number; sosActive: boolean;
  createdAt: string; updatedAt: string;
  // expanded on GET /jobs/:id
  vehicle?: Vehicle; mechanic?: Pick<User, 'id' | 'fullName' | 'phone' | 'garageName'> & { rating: number };
  checklist?: ChecklistItem[]; quotes?: PartsQuote[];
  totals?: { serviceFee: number; approvedParts: number; total: number };
}
export interface ChecklistItem { // job_checklists
  id: number; jobId: number; taskDescription: string;
  isCompleted: boolean; photoUrl: string | null; completedAt: string | null;
}
export interface PartsQuote { // parts_quotes
  id: number; jobId: number; partName: string; price: number;
  photos: string[]; // from photo_evidence
  isApproved: boolean | null; // null = awaiting owner
  createdAt: string;
}
export interface Review { // reviews
  id: number; jobId: number; mechanicId: number; ownerId: number;
  rating: 1 | 2 | 3 | 4 | 5; feedback: string | null; createdAt: string;
}
```

A.2 Payload examples

```
// POST /api/v1/sos
{ "vehicleId": 1, "issue": "Flat Tire", "lat": 0.3136, "lng": 32.5811 }
// 201 Created
{ "jobId": 42, "status": "pending", "nearbyCount": 3, "eta": "8 – 12 minutes" }

// GET /api/v1/jobs/1 (abridged)
{ "id": 1, "serviceType": "Brake Repair", "status": "fixing", "sosActive": false,
  "vehicle": { "make": "Toyota", "model": "Premio", "plateNumber": "UBK 482X" },
  "mechanic": { "id": 2, "fullName": "Joseph Okello", "phone": "+256 701 333444", "rating": 4.9 },
  "checklist": [ { "id": 1, "taskDescription": "Oil Level Checked", "isCompleted": true } ],
  "quotes": [ { "id": 2, "partName": "Brake Disc Rotor", "price": 210000, "isApproved": null,
    "photos": [ "https://cdn.../parts/part-1777453451335.jpeg" ] } ],
  "totals": { "serviceFee": 50000, "approvedParts": 120000, "total": 170000 } }

// Error envelope (all endpoints)
{ "error": { "code": "JOB_ALREADY_TAKEN", "message": "Another mechanic accepted this job." } } // HTTP
409
```

A.3 Existing enumerations to reuse

List	Values (from the web code)
SOS issues	Flat Tire, Dead Battery, Engine Failure, Towing, Low Fuel, Car Crash
Booking services	Oil Change, Brake Repair, General Service, Tyre Rotation
Diagnostic symptoms	Engine Light, Strange Noise, Overheating, Unusual Smoke
Fuel types	Petrol, Diesel, Hybrid, Electric
Transmission	Automatic, Manual
Booking checklist (10)	Oil Level Checked, Oil Filter Replaced, Coolant Level Checked, Tire Pressure Checked, Brake Pads Inspected, Battery Terminals Inspected, Lights & Signals Tested, Windshield Wipers Checked, Cabin Air Filter Checked, Under-Carriage Inspection
Arrival checklist (4)	Initial Inspection, Fluid Level Check, Diagnostic Scan, Safety Test
Job status	pending, accepted, diagnosing, fixing, ready, completed, cancelled
User status	active, pending, suspended

A.4 app.config.ts essentials

```
export default {
  name: 'MyCarRepair', slug: 'mycarrepair', scheme: 'mycarrepair',
  android: { package: 'ug.mycarrepair.app',
    permissions: ['ACCESS_FINE_LOCATION', 'ACCESS_BACKGROUND_LOCATION', 'CAMERA',
      'POST_NOTIFICATIONS', 'USE_FULL_SCREEN_INTENT', 'FOREGROUND_SERVICE_LOCATION'],
    config: { googleMaps: { apiKey: process.env.MAPS_KEY } } },
  ios: { bundleIdentifier: 'ug.mycarrepair.app',
    infoPlist: { NSLocationWhenInUseUsageDescription: 'Send help to exactly where you are.',
      NSCameraUsageDescription: 'Photograph your car and parts as evidence.' } },
  extra: { apiUrl: process.env.API_URL },
};
```